

Evaluation Playbook

Tesla Stock Price Prediction - Rohan R J

How to Use This

This is your prep sheet for the Live Evaluation Session. It is not something you read aloud - it is what you study beforehand so you can explain the project in your own words and answer follow-up questions without freezing. Walk through it once end to end, then skim the Q&A section again right before your session.

1. The 30-Second Pitch

If the evaluator only lets you say one thing, say this: I built two deep learning models, SimpleRNN and LSTM, to predict Tesla's stock closing price 1, 5, and 10 days ahead. I tuned both with GridSearchCV, evaluated them with MSE, and found that the simpler SimpleRNN model actually beat LSTM at every horizon on this dataset - which goes against what you'd normally expect, and I can explain why.

2. Walkthrough Order (what to click through, in order)

- Open the notebook. Point at the title cell - your name and batch are right there.
- Section 2 (Data Preprocessing): show the missing-value check, and say the forward-fill reasoning out loud (see Q&A below - this is a favorite question).
- Scroll past the 4 EDA charts - name each one in one sentence (see section 4 below).
- Section 3 (Model Development): show the two build functions side by side, point out they're identical except for the SimpleRNN/LSTM layer - this is what makes the comparison fair.
- Section on GridSearchCV: explain in one sentence why KerasRegressor is needed (see Q&A).
- Section 4 (Evaluation): show results_df and the actual-vs-predicted charts. This is where you deliver the main finding.
- Section 5 (Insights): this is basically your script for the finding + limitations + improvements. You can read from it if needed, that's fine.

3. The One Finding You Must Be Able to Explain

SimpleRNN beat LSTM at every prediction horizon (1, 5, and 10 days) in your final run. This surprises people because LSTM has a 'memory cell' specifically designed to remember longer patterns, so the textbook expectation is that LSTM wins, especially on longer horizons.

Your explanation: LSTM has more internal parameters than SimpleRNN (roughly 4x more per unit, since it has to learn gates for what to remember, forget, and output). More parameters need more data to learn well. Your training set is about 1,900 sequences - reasonably small for a deep learning problem. With that little data, LSTM's extra complexity doesn't have enough signal to pay off, so the simpler SimpleRNN generalizes better instead of overfitting. This is a real, well-known pattern in machine learning: bigger models need more data, and when you don't have much data, the simpler model can win.

4. One-Sentence Summary for Each Chart

- Close price line chart: shows Tesla's raw price trend from 2010 to 2020, mostly flat then rising sharply

after 2013.

- Moving averages chart: the 50-day and 200-day averages smooth out daily noise so you can see the underlying trend more clearly.
- Volume bar chart: shows how many shares traded per day - spikes usually line up with major news or earnings events.
- Correlation heatmap: shows Open, High, Low, Close, and Adj Close are almost perfectly correlated with each other, which is why using just Close as the target is reasonable.
- Actual vs predicted charts (one per horizon): show how closely each model's predictions track the real price on data the model never saw during training.

5. Q&A - Questions You Are Likely to Get

Q: Why did you use forward-fill instead of filling missing values with the mean or median?

A: Because this is time-series data. The mean price across 10 years ignores whether the stock was trending up or down at that specific point. Forward-fill carries the last known price forward, which is a much more realistic guess for a single missing trading day, and it never uses future information, which would be cheating.

Q: Why did you scale the data, and why fit the scaler only on training data?

A: Neural networks train faster and more stably when inputs are small numbers (0 to 1) instead of raw dollar values in the hundreds. I fit MinMaxScaler only on the training set and then applied that same fitted scaler to the test set, because fitting on the whole dataset would leak information about future test-set prices into training - the model would effectively know the future min and max price in advance.

Q: Why didn't you shuffle the data before splitting into train and test?

A: Because order matters here. Shuffling would let the model train on data that comes after the test period in time, which is exactly the kind of future information it should never have access to. I split chronologically instead - oldest 80 percent for training, most recent 20 percent for testing - so the test set genuinely represents unseen future data.

Q: What is the 60-day window for?

A: Each prediction is based on the past 60 trading days of Close price. So to predict tomorrow's price, the model looks at the last 60 days and learns the pattern in that sequence. I chose 60 as a reasonable amount of recent history, roughly 3 trading months.

Q: What's the difference between your 1-day, 5-day, and 10-day models?

A: They're separate models trained on the same 60-day input window, but with different targets: the 1-day model predicts tomorrow's price, the 5-day model predicts the price 5 trading days from now, and the 10-day model predicts 10 days out. I did not chain predictions forward - each horizon has its own dedicated model trained directly on that target.

Q: Why do you need KerasRegressor / scikeras for GridSearchCV?

A: GridSearchCV is a scikit-learn tool, and it expects models with a scikit-learn-style interface (fit, predict, get_params). Keras models don't have that interface natively, so KerasRegressor from the scikeras library wraps my Keras model to make it look like a scikit-learn estimator, so GridSearchCV can call it.

Q: What hyperparameters did you tune, and what did GridSearchCV find?

A: I tuned the number of units in the recurrent layer, the dropout rate, and the learning rate. For both models, GridSearchCV found 32 units, a dropout rate of 0.2, and a learning rate of 0.01 worked best on the 1-day-horizon data - I then reused those settings for the 5-day and 10-day models of the same architecture to keep the search fast.

Q: What do EarlyStopping and ModelCheckpoint actually do?

A: EarlyStopping watches the validation loss during training and stops automatically once it stops improving, restoring the best weights seen - this prevents the model from overfitting by training too long. ModelCheckpoint saves the best-performing version of the model to disk as it trains, so I always have the best model saved even if training continues past that point.

Q: Why MSE instead of accuracy?

A: Accuracy is for classification problems with discrete categories. This is a regression problem - I'm predicting a continuous dollar value - so I need a metric that measures how far off my numeric predictions are. Mean Squared Error does that: it averages the squared difference between predicted and actual price, and squaring penalizes big misses more than small ones.

Q: Why did SimpleRNN outperform LSTM here? Isn't LSTM supposed to be better?

A: LSTM is usually better on longer sequences and larger datasets because its memory cell can learn what to remember and forget. But LSTM has roughly 4x more parameters than SimpleRNN for the same number of units, and my training set is only about 1,900 sequences - fairly small for deep learning. With that little data, LSTM's extra capacity doesn't have enough signal to learn from properly, so it doesn't generalize as well, while the simpler SimpleRNN does fine with less to learn. This is a real and defensible finding, not a mistake.

Q: Why does error increase from the 1-day to the 10-day model?

A: Because predicting further into the future is inherently harder - small uncertainties compound the further out you go. My results show MSE growing roughly 5 to 7 times from the 1-day horizon to the 10-day horizon for both models, which matches that expectation.

Q: What are the biggest limitations of this model?

A: It only uses the Close price as input, so it has no way to react to news, earnings reports, or other real-world events that actually move the stock - it can only extrapolate from recent price patterns. It also was tuned on the 1-day horizon only and reused for 5 and 10 days, so a separate tuning pass per horizon might change the results. And performance on the 2018-2020 test period isn't guaranteed to hold in future, different market conditions.

Q: How would you improve this project further?

A: Add more input features like trading Volume, news sentiment, or macroeconomic indicators such as interest rates. Compare against other architectures like GRU or a classical ARIMA baseline. Run GridSearchCV separately for each horizon instead of reusing the 1-day result. And apply the same pipeline to competitor stocks like Rivian or NIO for comparison.

Q: If you ran this notebook again, would you get the exact same numbers?

A: Not exactly the same, no - training a neural network involves randomness in weight initialization and dropout that a single random seed doesn't fully control in TensorFlow. But the overall pattern has held consistently across my runs: error grows with horizon, and SimpleRNN performs competitively with or better than LSTM on this dataset.

6. If You Freeze

Fall back to this sentence and keep going: 'I built and compared two recurrent neural network types on Tesla's closing price, tuned them with GridSearchCV, and found the simpler model actually performed better here because my dataset is relatively small for deep learning.' That sentence alone demonstrates you understand the core of the project even if you blank on a specific number.